VUE.JS

UTILISATION D'UN FRAMEWORK

VUE.JS

Pourquoi un framework ?

Le HTML / CSS / JavaScript respecte des standards évoluants peu.

Le développement d'applications web a besoin de techniques / logiques modernes et modulables.

Pensez à l'ex. 6 (sliders) et imaginez créer une application web comme Discord, YouTube, Spotify, un géoportail, etc

Un framework est un ensemble de fonctions / modules / outils pour faciliter le développement.

Cela simplifie les tâches fastidieuses et organise mieux le code.

VUE.JS

Installation

Vue.js peut être utilisé comme librairie simple (fichier JS a inclure dans votre page), ou comme module de NodeJS.

Dans les exercices, l'installation se fera avec NodeJS.

VUE.JS

Première application

L'application Vue.js de base est :

```
<div id="app"></div>

<script type="module">
import { createApp } from 'vue'

const app = createApp(/* élément racine de l'application */)
app.mount('#app')
</script>
```

L'élément `<div>` est un élément HTML div standard

Le code JavaScript est placé dans une balise script qui est un module, comme d'habitude

VUE.JS

Première application

L'application Vue.js de base est :

```
<div id="app"></div>

<script type="module">
import { createApp } from 'vue'

const app = createApp(/* élément racine de l'application */)
app.mount('#app')
</script>
```

La fonction `createApp` est la fonction de base de Vue. Elle prend en paramètre l'élément racine de l'application.
La fonction `mount` de l'application crée l'application (il ne se passe rien avant). Elle prend en paramètre l'élément HTML dans lequel créer l'application, ici l'élément avec l'id "app".

VUE.JS

Première application

L'application Vue.js de base est :

```
<div id="app"></div>

<script type="module">
import { createApp } from 'vue'
import App from './App.vue'

const app = createApp(App)
app.mount('#app')
</script>
```

Généralement, on déclare l'élément racine dans un fichier séparé, qu'on inclus.

Le contenu de la balise `script` est souvent aussi placé dans un fichier "main.js"

VUE.JS

Première application

Les composants de Vue sont généralement placés dans des fichiers .vue

```
// Fichier App.vue

<script></script>

<template>Contenu du composant</template>

<style scoped></style>
```

On trouve trois éléments dans la définition d'un composant

1. Une balise `script` qui définit la logique du composant et son initialisation
2. Une balise `template` qui définit le code HTML du composant
3. Une balise `style` qui définit le style propre au composant

VUE.JS

Première application

```
<script>
  export default {
    data() { // Déclaration de l'état du composant et de ses propriétés
      return { count: 0 }
    },
    methods: { // Méthodes propres au composant
      increment() { this.count++ }
    },
    mounted() { /* script d'initialisation */ }
  }
</script>
<template>Contenu du composant</template>
<style scoped></style>
```

Dans la balise script, on peut trouver plusieurs définitions dont

- `data()` qui retourne un objet contenant les propriétés du composant
- `methods` qui retourne un objet contenant les méthodes du composant
- `mounted()` qui est une fonction exécutée quand le composant est "monté" (ajouté au HTML de la page)

VUE.JS

Première application

```
<script>
  export default {
    data() {
      return { count: 0 }
    },
    methods: {
      increment() { this.count++ }
    },
    mounted() { }
  }
</script>
<template>
  <p>{{count}}</p>
  <button v-on:click="increment">Incrémenter</button>
</template>
<style scoped></style>
```

On peut utiliser des propriétés dans le HTML avec la syntaxe `{{ prop }}`

`v-on` est ce qu'on appelle une *directive*, c'est un élément spécifique à Vue qui décrit un comportement particulier, ici une méthode à appeler au clic sur le bouton.

VUE.JS

Première application

```
<script>
  export default {
    data() {
      return { count: 0 }
    },
    methods: {
      increment() { this.count++ }
    },
    mounted() { }
  }
</script>
<template>
  <p>{{count}}</p>
  <button v-on:click="increment">Incrémenter</button>
</template>
<style scoped></style>
```

Au centre de Vue se trouve le principe de *réactivité*, ici `count` est une propriété qui, quand elle est modifiée, va automatiquement mettre à jour tous les éléments qui en dépendent.

Cliquer sur le bouton va incrémenter `count` et mettre à jour le contenu de la balise `p`, automatiquement.

VUE.JS

Première application

```
<script>
  export default {
    data() {
      return { displayed: true }
    },
    methods: {
      toggleDisplay() { this.displayed=!this.displayed }
    },
    mounted() { }
  }
</script>
<template>
  <p v-if="displayed" >Paragraphe affiché</p>
  <button @click="toggleDisplay">Afficher / Cacher le paragraphe</button>
</template>
<style scoped></style>
```

La directive `v-if` permet d'indiquer si un élément doit être affiché ou non

La directive `v-on` peut être remplacée par `@`, qui est plus court

VUE.JS

Les *computed properties*

```
<script>
  export default {
    data() {
      return { age: 5 }
    },
    computed: {
      isAdult() { return this.age >= 18}
    },
    methods: { },
    mounted() { }
  }
</script>
<template>
  <p>Age : {{age}}, {{isAdult ? 'Majeur' : 'Mineur'}}</p>
  <button @click="this.age+=1">+</button>
  <button @click="this.age-=1">-</button>
</template>
<style scoped></style>
```

Dans `computed` on peut indiquer des propriétés qui dépendent d'autres.

Dans des `{{ ... }}` il est possible de mettre des ternaires (`condition` ? `siVrai` : `siFaux`)

VUE.JS

Les *computed properties*

```
<script>
  export default {
    data() {
      return { age: 5 }
    },
    computed: {
      isAdult() { return this.age >= 18}
    },
    methods: { },
    mounted() { }
  }
</script>
<template>
  <p>Age : {{age}}, {{isAdult ? 'Majeur' : 'Mineur'}}</p>
  <button @click="this.age+=1">+</button>
  <button @click="this.age-=1">-</button>
</template>
<style scoped></style>
```

La différence entre les *computed properties* et les méthodes est que la valeur des *computed properties* est mise en cache. Elle n'est recalculée que si une propriétés dont dépend la *computed property* change.